CS-3180: Comparative Languages Lab Problem 6: Mastermind

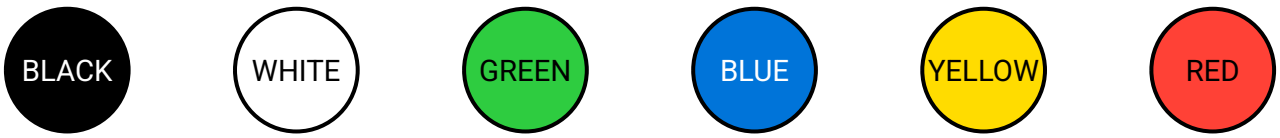
PURPOSE: The goal of this lab is to program an interactive implementation of Mastermind in Lisp. This lab will combine string processing and list manipulation. You will also handle state tracking along with interactive terminal I/O.

DIRECTIONS: You will implement your program in a file named **`Mastermind.lisp`**. will must generate a random 4-peg secret sequence from six allowed colors and prompt the user for space-separated input guess. You will score guesses using standard Mastermind feedback rules ("BLACK" for exact position/color matches, "WHITE" for correct color/wrong position), and continue until the player wins.

To get a feel for the "call and response" gameplay, I would highly encourage you to play around with the game a bit, before you begin implementing the project. An online version can be played at https://www.master-mind.dk/. I also have a physical copy that can be played in my office, or after class if I remember to bring it. It should feel similar to an "Algorithmic Wordle."

Rules:

- **Color Palette**: Valid pegs are the strings **`"BLACK"`**, **`"WHITE"`**, **`"GREEN"`**, **`"BLUE"`**, **`"YELLOW"`**, and **`"RED"`**.



Scoring: Convert user input to uppercase and split space-separated colors into a list. If a guess does not contain exactly 4 pegs you should display an error message.

Feedback Rules:

- Respond with a **`"BLACK"`** feedback peg when a correct color is in the exact correct position.
- Respond with a **`"WHITE"`** feedback peg when a correct color is in the wrong position.
- Once a peg in the secret sequence is matched (either exact or color-only), it cannot be matched again during that same round. This is prevent double counting of pegs in a round.



EXAMPLE: Sample Game

```
> ./lisp.sh Mastermind.lisp
Enter a guess:
white blue blue blue
("BLACK")
Enter a guess:
white red white yellow
NIL
Enter a guess:
```

```
black red red red blue
Error: please only select 4 pegs
Enter a guess:
black blue black black
("WHITE")
Enter a guess:
green green blue green
You win!
```

(Note: The order of feedback items within the result list may vary depending on your scoring implementation, provided the counts are accurate. For this program, you can assume that a response of ("BLACK" "WHITE") is equivalent to ("WHITE" "BLACK"))

For random value generation (e.g code-generation), you will need to use the **`random`** function in Lisp, as well as having the following line at the top of your program, to ensure different random values each execution.

```
(setf *random-state* (make-random-state t))
```

RUBRIC:

(85 pts) **Base Functionality**

[10] Secret generation selects valid 4-peg sequences randomly

[20] Parse input, converts string, and handle input case-insensitivity, with 4-peg limit

[40] Evaluation function(s) correctly calculates correct ("BLACK") and partial ("WHITE") matches without double counting

[15] Driver program manages state, displays feedback, and handles win conditions

(15 pts) **Style**

[5] Clean project structure

[10] Completed lab report in Markdown format

IMPORTANT NOTES:

- **Submissions that do not compile will receive a zero**
- **Submissions with improperly cited AI will receive a zero and an academic integrity violation**
- **Submissions that are partially or fully copied from another submission will receive a zero and an academic integrity violation**